



Cook'île & Co

Estelle Cook'île & Co

DOSSIER PROJET

Titre Professionnel Développeur Web et Web Mobile : RNCP Niveau 5

Candidat	Anthony Pailloux
Établissement	La Plateforme
Titre visé	Développeur Web et Web Mobile (DWWM)
Année	2025-2026
Projet	Application web : catalogue de recettes antillaises et du monde
Client	Estelle
URL	https://www.cookileandco.fr
Accroche	<i>An Nou Ay !</i>

Stack : React, Vite, Node.js, Express, Sequelize, MySQL, hébergement o2switch.

Sommaire

Résumé du projet	p. 4
Liste des compétences couvertes	p. 5
1. Présentation et expression des besoins	p. 6
1.1 Contexte et genèse du besoin	p. 6
1.2 Concept et objectifs	p. 6
1.3 Positionnement et différenciants	p. 6
1.4 Public cible et acteurs	p. 7
1.5 Besoins fonctionnels	p. 7
1.6 Besoins non-fonctionnels et contraintes	p. 8
1.7 Gestion de projet	p. 8
2. Spécifications d'interface et conception	p. 10
2.1 Arborescence des écrans	p. 10
2.2 Maquettes et wireframes	p. 10
2.3 Charte graphique (thème Madras)	p. 16
2.4 Responsive design	p. 16
2.5 Modélisation de la base de données (MCD)	p. 17
2.6 Lien avec les migrations Sequelize	p. 19
3. Architecture et environnement technique	p. 21
3.1 Architecture générale	p. 21
3.2 Organisation du front-end	p. 22
3.3 Organisation du back-end	p. 22
3.4 Stack technique et justifications	p. 23
3.5 API REST (aperçu)	p. 23
3.6 Sécurité	p. 24
3.7 Hébergement et déploiement (o2switch)	p. 24
4. Réalisations	p. 26
4.1 Réalisations back-end	p. 26
4.2 Réalisations front-end	p. 29
4.3 Preuves de test API (Postman)	p. 30
4.4 Compétences illustrées	p. 32
5. Jeu d'essai et stratégie de test	p. 33
5.1 Contexte du scénario	p. 33
5.2 Tableau de test	p. 33
5.3 Illustrations du scénario	p. 33
5.4 Données observées	p. 36
5.5 Conclusion du jeu d'essai	p. 36
6. Veille technologique et sécurité	p. 37
6.1 Comment j'ai mené la veille	p. 37
6.2 Sources utilisées	p. 37
6.3 Sécurisation des sessions admin	p. 37
6.4 Validation et upload	p. 39
6.5 Bilan	p. 39
7. Conclusion et bilan	p. 40

7.1 En bref	p. 40
7.2 Ce qui est en place	p. 40
7.3 Difficultés rencontrées	p. 40
7.4 Ce que j'ai appris	p. 40
7.5 Limites du projet	p. 40
7.6 Perspectives	p. 41
7.7 Conclusion personnelle	p. 41
Annexes	p. 42
Lexique technique	p. 42

Résumé du projet

Cook'île & Co est un site web dédié à la cuisine de Estelle, passionnée par les saveurs des Antilles et du monde. L'objectif est de rassembler ses recettes familiales dans un catalogue en ligne accessible à tous, sans création de compte pour les visiteurs.

Sur le site, chacun peut parcourir les recettes, les filtrer par origine, type de repas ou catégorie, et consulter une fiche détaillée avec photo, liste des ingrédients, étapes de préparation et conseils de Estelle. L'ambiance visuelle s'inspire du tissu madras et des couleurs chaudes des Antilles.

Estelle et le développeur disposent d'un espace réservé pour ajouter, modifier ou supprimer les recettes, gérer les catégories et les photos. Cet espace est accessible uniquement après une connexion sécurisée (2 comptes administrateurs).

L'idée est de garder ces recettes de famille accessibles en ligne, pour Estelle comme pour ceux qui veulent découvrir la cuisine antillaise et d'ailleurs.

Liste des compétences couvertes

Compétences du référentiel **front-end / back-end d'une application web sécurisée**, mobilisées sur Cook'île & Co.

Réf.	Compétence	Voir section	Illustré par
1.1	Installer et configurer son environnement de travail	§ 1.7 · § 3	Laragon, Node.js, MySQL, <code>.env</code> , scripts npm
1.2	Maquetter des interfaces utilisateur	§ 2.2	Maquettes Figma, wireframe → maquette
1.3	Réaliser des interfaces utilisateur statiques	§ 4.2	Composants React, CSS thème Madras
1.4	Développer la partie dynamique des interfaces	§ 4.2	<code>fetch</code> , état React, filtres, AuthContext
2.5	Mettre en place une base de données relationnelle	§ 2.5	MCD Excalidraw, dictionnaire, phpMyAdmin
2.6	Développer des composants d'accès aux données (ORM)	§ 3 · § 4.1	Sequelize, migrations, modèles
2.7	Développer des composants métier côté serveur	§ 4.1 · § 4.3	API REST, validation, <code>requireAdmin</code> , Postman
2.8	Documenter le déploiement d'une application dynamique	§ 3.7	Procédure o2switch · https://www.cookileandco.fr

1. Présentation et expression des besoins

1.1 Contexte et genèse du besoin

Estelle partage depuis longtemps ses recettes de cuisine antillaise et du monde avec son entourage. Ces recettes existaient surtout de façon dispersée : carnets, notes, échanges oraux. Il n'existait pas d'endroit unique, moderne et agréable pour les consulter et les faire vivre.

Cook'île & Co répond à ce besoin : un **catalogue en ligne** chaleureux où l'on consulte les recettes sans compte, et où la cliente peut les ajouter ou les corriger simplement.

C'est mon **Projet de Fin d'Année** Développeur Web et Web Mobile : de la conception à la mise en ligne, en passant par le code et la documentation.

1.2 Concept et objectifs

Cook'île & Co est un site de recettes qui met en avant les saveurs des Antilles et du monde, avec une identité visuelle inspirée du **tissu madras**.

Objectif	Description
Catalogue public	Liste paginée des recettes, recherche et filtres
Fiche recette	Photo, temps, ingrédients structurés, étapes, conseils de Estelle
Référentiels	Catégories, origines, types de repas
Espace admin	Connexion sécurisée, CRUD recettes et référentiels
Identité visuelle	Thème Madras, logo, accroche <i>An Nou Ay !</i>
Mise en ligne	Site accessible sur https://www.cookileandco.fr

1.3 Positionnement et différenciants

Marmiton, les réseaux sociaux ou cuisine-creole.com permettent de trouver des recettes, mais pas de publier et gérer ses fiches comme Estelle le souhaitait. Voici où se situe Cook'île & Co :

Solution existante	Principe	Limite
Carnets / notes papier	Recettes manuscrites ou orales	Difficiles à partager, pas de recherche
Réseaux sociaux	Photos et posts ponctuels	Pas de structure (ingrédients, étapes, filtres)
Grands sites (Marmiton, 750g...)	Catalogue généraliste	Recettes de tous les auteurs : Estelle n'y publie pas son propre livre de recettes
Cuisine créole (cuisine-creole.com)	Recettes antillaises et créoles en ligne	Bonne source d'inspiration, mais ce ne sont pas ses recettes et elle ne peut pas les gérer

Ce qui distingue Cook'île & Co :

- Un **catalogue dédié** aux recettes d'Estelle, pas un catalogue multi-auteurs.
- Consultation **libre** pour les visiteurs (pas d'inscription obligatoire).
- Fiches structurées : quantité · unité · nom, étapes numérotées, conseils.
- Filtres métier : **origine, type de repas, catégorie**.
- Back-office réservé aux **administrateurs** (Estelle et le développeur).
- Ambiance visuelle **Madras** (couleurs chaudes, marque claire).

1.4 Public cible et acteurs

Personas

Persona 1 : Léa, 32 ans, visiteuse curieuse

Léa aime la cuisine du monde et cherche des recettes antillaises fiables. Elle consulte surtout sur téléphone, veut filtrer rapidement (origine, type de repas) et ouvrir une fiche claire avec photo et étapes, **sans créer de compte**.

Persona 2 : Estelle, porteuse de contenu

Estelle veut publier et corriger ses recettes elle-même, ajouter des photos, classer par catégorie / origine / type de repas. Elle n'est pas développeuse : l'interface admin doit rester simple. Deux comptes admin (Estelle et le développeur) peuvent gérer le catalogue.

Acteurs du système

Acteur	Description	Droits principaux
Visiteur	Internaute non connecté	Accueil, catalogue, fiches, catégories ; recherche et filtres
Administrateur(s)	Estelle et le développeur (2 comptes admin)	Connexion ; CRUD recettes (photo, ingrédients, étapes) ; gérer catégories, origines, types de repas

1.5 Besoins fonctionnels

- Consulter le catalogue de recettes (liste paginée, recherche par titre).
- Consulter une fiche détaillée (photo, temps, ingrédients, étapes, conseils).
- Filtrer par origine, type de repas et catégorie.
- Parcourir les pages catégories / origines.
- Authentification admin (email + mot de passe, session sécurisée).
- CRUD recettes côté admin (création, modification, suppression, upload photo).
- Gérer les référentiels (catégories, origines, types de repas).
- Déployer l'application en production (HTTPS, domaine dédié).

1.6 Besoins non-fonctionnels et contraintes

Type	Exigence
Sécurité / conformité	Mots de passe hashés (bcrypt) ; routes admin protégées ; cookie de session httpOnly ; validation serveur
Ergonomie	Interface responsive (mobile / desktop), thème Madras lisible
Performance	Catalogue paginé ; images WebP/JPEG/PNG limitées à l'upload
Maintenabilité	API REST structurée (routes / contrôleurs / middlewares / modèles)
Hébergement	o2switch, Node.js + MySQL

1.7 Gestion de projet

Méthodologie

J'ai développé l'application fonctionnalité par fonctionnalité, en contrôlant chaque ajout en local sur Laragon. Le déploiement en production a été réalisé à la fin, suivi de quelques correctifs d'ajustement. Les tests en production ont été effectués par la cliente et par moi-même.

Planning macro

Phase	Contenu
Cadrage	Cahier des charges, MCD, maquettes, stack
MVP	Accueil, catalogue, fiche, auth admin, CRUD
Enrichissement	Filtres, référentiels, upload photos, responsive
Mise en ligne	o2switch, HTTPS, vérifications post-déploiement
Dossier	Maquettes, rédaction, jeu d'essai, veille

Suivi avec Git

Versionnement sur GitHub : branches / commits par fonctionnalité, déploiement prod via `git pull` sur le serveur o2switch.

Outils

Outil	Usage
Laragon	MySQL, phpMyAdmin, environnement local Windows
VS Code	Édition du code
Figma	Maquettes
Excalidraw	MCD
Postman	Tests API (collection <i>cookîle&co</i>)
Git / GitHub	Versionnement et déploiement

Outil	Usage
o2switch	Hébergement production

2. Spécifications d'interface et conception

2.1 Arborescence des écrans

Voici la structure des pages telle que je l'ai définie dans Figma :

Page	URL	Accès
Accueil	/	Public
Catalogue	/recettes	Public
Fiche recette	/recettes/:id	Public
Catégories & origines	/categories	Public
Connexion admin	/connexion-superadmin	Admin
Dashboard	/admin	Admin

Parcours utilisateurs

Visiteur : Accueil → Catalogue (filtres) → Fiche recette → (partage éventuel).

Administrateur(s) : Connexion → Dashboard → Créer / modifier une recette → Contrôle sur le catalogue public.

2.2 Maquettes et wireframes

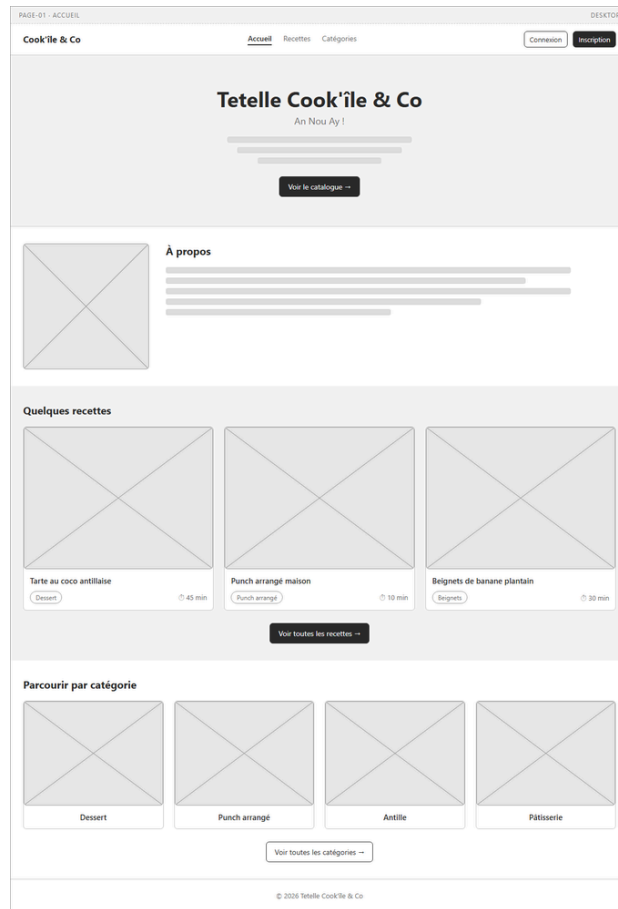
J'ai commencé par regarder Marmiton et 750g pour m'inspirer de leur organisation : hero d'accueil, grille de recettes, fiche détaillée, puis l'espace admin. Tout a été maqueté sur **Figma** avant de coder.

Écran	Contenu principal
Accueil	Hero, accroche <i>An Nou Ay !</i> , accès au catalogue
Catalogue	Grille de recettes + filtres origine / type / catégorie
Fiche recette	Photo, ingrédients, étapes, conseils de Estelle
Connexion admin	Formulaire email / mot de passe
Dashboard admin	Liste des recettes, actions CRUD

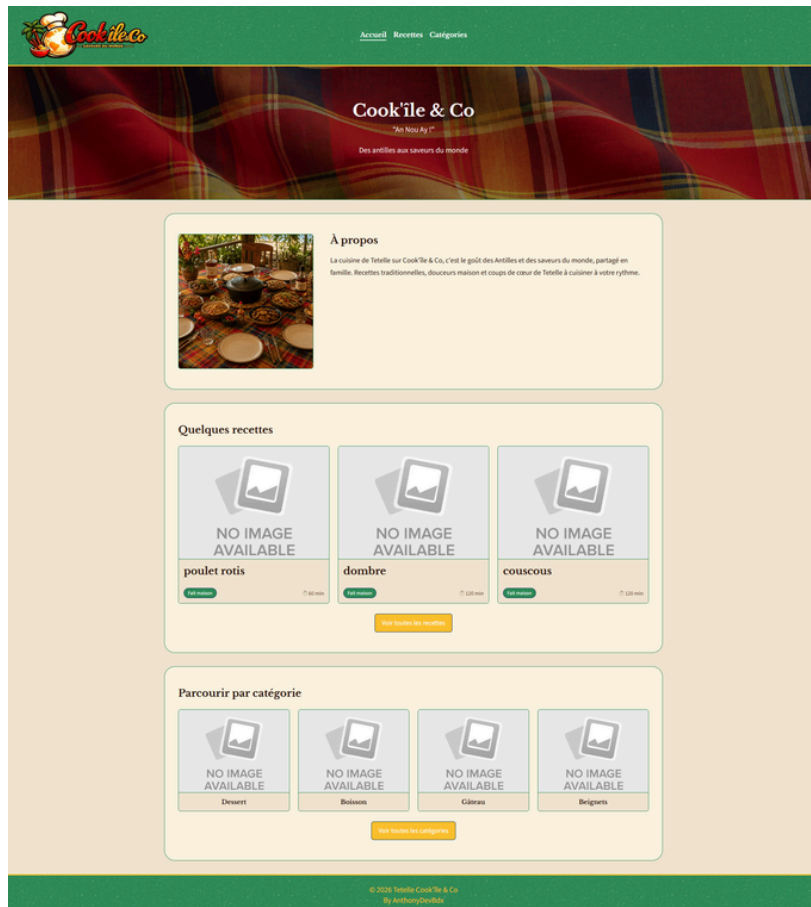
Les écrans développés reprennent cette structure, enrichie du **thème Madras** (couleurs, motifs).

Écrans Figma : wireframe et maquette

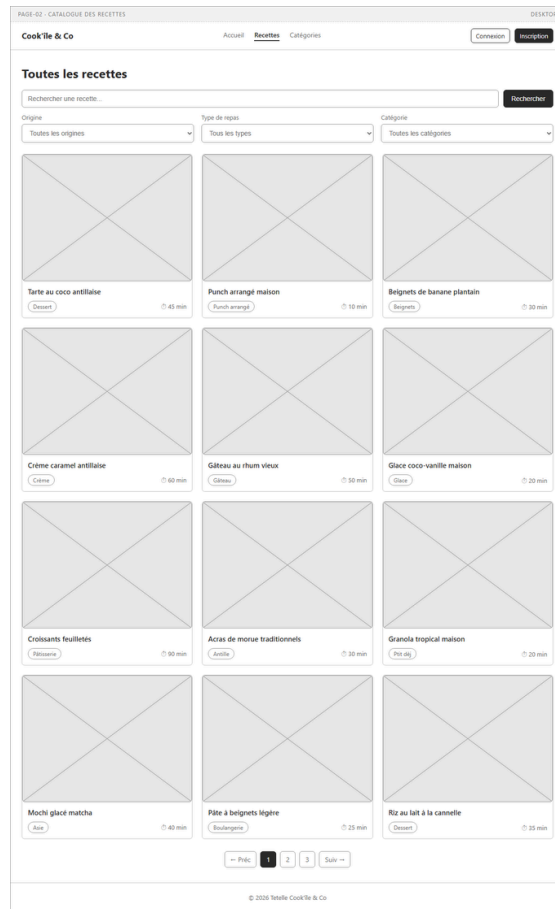
Accueil : Wireframe Figma



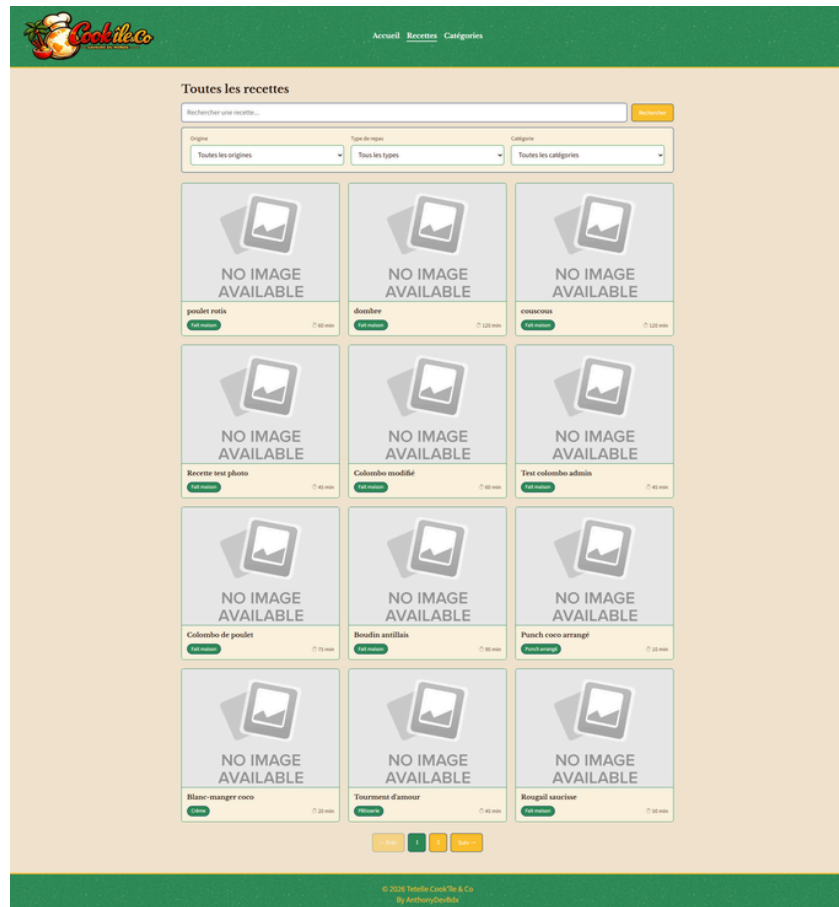
Accueil : Maquette Figma



Catalogue : Wireframe Figma



Catalogue : Maquette Figma



Fiche recette : Wireframe Figma

PAGE-03 - FICHE RECETTE DESKTOP

Cook'ile & Co Accueil Recettes Catégories Connexion Inscription



Tarte au coco antillaise

Dessert

45 minutes

Partager

Ingrédients

-
-
-
-
-
-
-
-

Préparation

-
-
-
-
-
-

Conseils de Tetelle

Partager cette recette

Facebook · WhatsApp · Copier le lien

© 2026 Tetelle Cook'ile & Co

Fiche recette : Maquette Figma

 Accueil Recettes Catégories



poulet rotis

448 calories

45 minutes

Partager cette recette

Facebook · WhatsApp · Copier le lien

Ingrédients

- 2 kg poulet

Préparation

- Préchauffer votre four à 200°

Conseils de Tetelle

Le recouvert de papier d'aluminium

Commentaires

Votre pseudo

Ex: Marie, Christophe

Votre commentaire

Partagez votre avis sur cette recette...

Verification anti-spam

6 + 7 = ? Répondre

Recevez l'adresse pour commenter que si le site est valide.

Publish

Laetitia

7 adams

Stéphane

C'est la meilleure recette.

Robert

super merci.

Tony

Délicieux !

© 2026 Tetelle Cook'ile & Co
By AnthonyPailloux

Connexion admin : Wireframe Figma

PAGE-05 - CONNEXION DESKTOP

Cook'île & Co

Se connecter

Email

Mot de passe

Se connecter

Pas encore de compte ? [S'inscrire](#)

© 2026 Tetelle Cook'île & Co

Connexion admin : Maquette Figma



Accueil Recettes Catégories

Formulaire de connexion

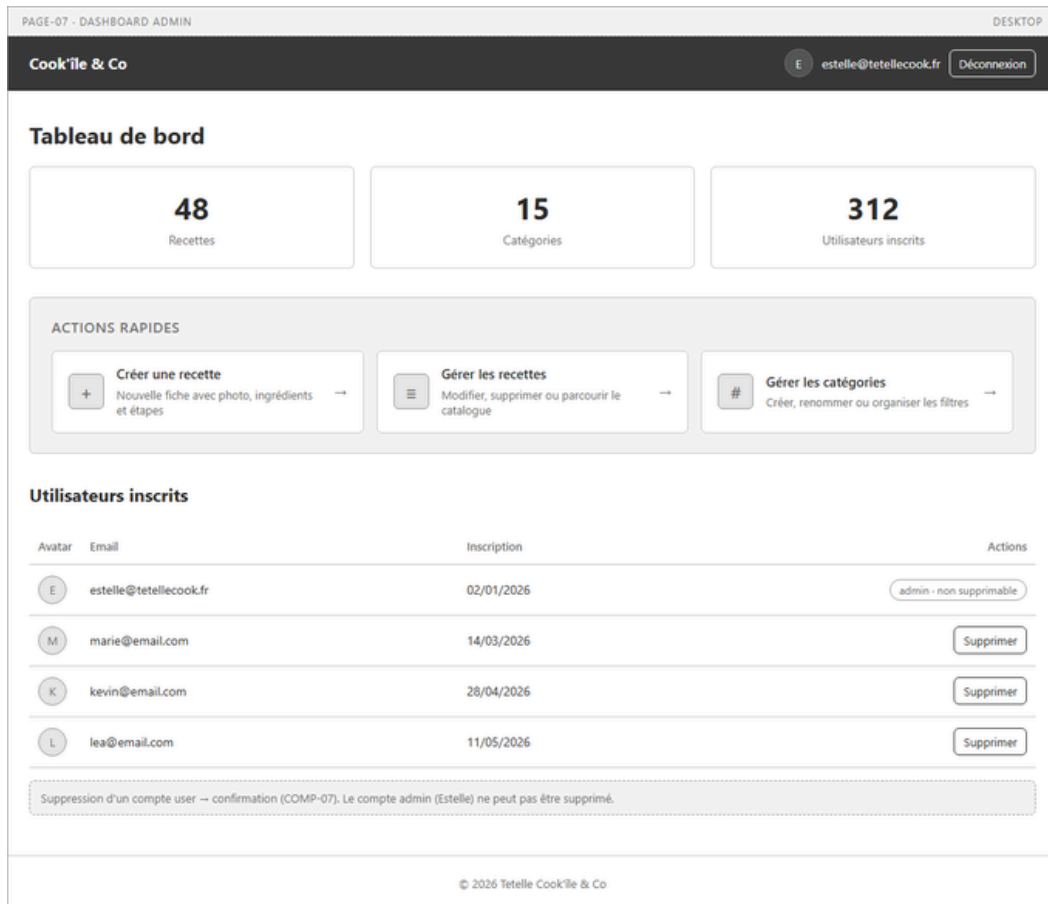
Email*

Mot de passe*

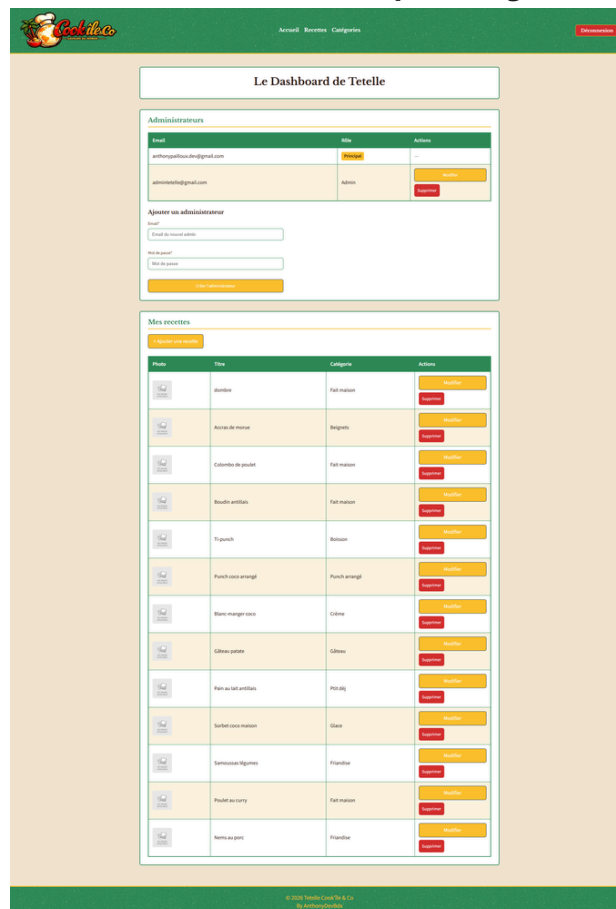
Se connecter

© 2026 Tetelle Cook'île & Co
By AnthonyDevBdx

Dashboard admin : Wireframe Figma



Dashboard admin : Maquette Figma



2.3 Charte graphique (thème Madras)

Élément	Choix	Pourquoi
Univers	Antilles, madras, saveurs du monde	Identité Estelle / Cook'île
Couleurs	Rouge, orange, jaune, vert ; bleu marine marque	Chaleur + contraste header/footer
Fond	Beige / crème	Lisibilité des cartes recettes
Typographie	Libre Baskerville (titres) · Source Sans 3 (texte)	Titres chaleureux, corps de texte lisible
Logo	Cook'île & Co, « Saveurs du monde »	Reconnaissance de marque
Accroche	<i>An Nou Ay !</i>	Ancrage culturel

Les polices sont chargées via Google Fonts et déclarées dans `client/src/style/themes.css` :

Variable	Police	Usage
<code>--tc-font-heading</code>	Libre Baskerville	Titres, nom de marque
<code>--tc-font</code>	Source Sans 3	Paragraphes, menus, formulaires

Couleurs principales (`themes.css`) :

Variable	Couleur	Usage
<code>--tc-navy</code>	<code>#1b3a6b</code>	Bleu marine : header, marque
<code>--tc-red-madras</code>	<code>#d32f2f</code>	Rouge madras : boutons
<code>--tc-yellow-madras</code>	<code>#fbc02d</code>	Jaune : CTA secondaires
<code>--tc-green-madras</code>	<code>#2e8b57</code>	Vert : badges
<code>--tc-background</code>	<code>#f0e4d0</code>	Beige : fond principal

2.4 Responsive design

Pas de site mobile séparé : la mise en page s'adapte avec des grilles CSS et un menu simplifié sur petit écran.

- **Smartphone** : cartes empilées, filtres accessibles.
- **Tablette / desktop** : grilles multi-colonnes, header complet.

Maquettes mobile (Figma) :

Catalogue mobile



Accueil mobile



Fiche recette mobile



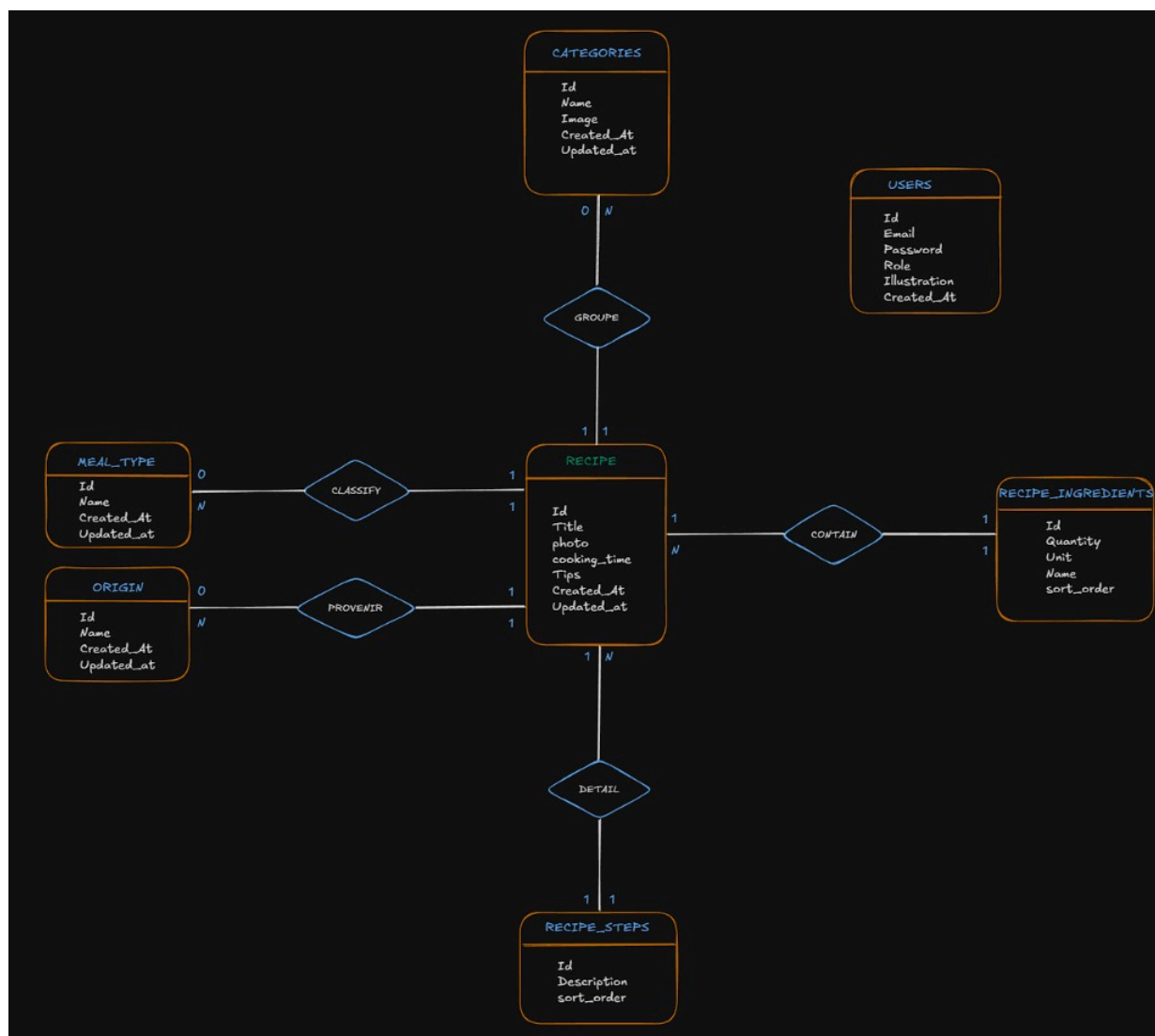
Catégories mobile



Principes retenus : peu d'actions par écran côté public ; retour visuel après action admin (toasts) ; photo bien visible sur les cartes recettes.

2.5 Modélisation de la base de données (MCD)

MCD réalisé sur **Excalidraw**.



Modèle conceptuel de données

Association	Entités	Cardinalités
GROUPE	CATEGORIES → RECIPE	(0,N) / (1,1)
CLASSIFY	MEAL_TYPE → RECIPE	(0,N) / (1,1)
PROVENIR	ORIGIN → RECIPE	(0,N) / (1,1)
CONTAIN	RECIPE → RECIPE_INGREDIENTS	(1,1) / (1,N)
DETAIL	RECIPE → RECIPE_STEPS	(1,1) / (1,N)

Règles métier : une recette a **une** catégorie, **une** origine et **un** type de repas ; plusieurs ingrédients et étapes ; le contenu est géré par un ou plusieurs comptes **admin**.

Dictionnaire de données

Table `recipes` :

Colonne	Type	Contrainte	Description
<code>id</code>	INT	PK	Identifiant

Colonne	Type	Contrainte	Description
title	VARCHAR	NOT NULL	Titre
photo	VARCHAR	NULL	Chemin upload
cooking_time	INT	NOT NULL	Minutes
tips	TEXT	NULL	Conseils Estelle
category_id	INT	FK	Catégorie
origin_id	INT	FK	Origine
meal_type_id	INT	FK	Type de repas

Autres tables :

Table	Colonnes clés	Rôle
users	email, password_hash, role	Compte admin
categories / origins / meal_types	name (+ image catégories)	Référentiels
recipe_ingredients	quantity, unit, name, sort_order	Ingrédients
recipe_steps	description, step_number, sort_order	Étapes

2.6 Lien avec les migrations Sequelize

Le schéma est versionné avec **Sequelize CLI** (`npm run db:migrate`). Les associations du modèle `Recipe` reflètent le MCD (`belongsTo` catégorie / origine / type de repas ; `hasMany` ingrédients / étapes).

`SELECT * FROM 'recipes'`

Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

[dra options](#)

☐ Check all With selected: Edit Copy Delete Export

3. Architecture et environnement technique

Front React, API Express, base MySQL : voici comment les morceaux s'assemblent et pourquoi j'ai fait ces choix.

3.1 Architecture générale

L'application est séparée en deux parties qui communiquent en JSON via une API REST :

Étape	Élément	Échange
1	Navigateur (React / Vite)	Interface utilisateur
2	<code>fetch</code> + cookie session (<code>credentials: include</code>)	Appels HTTP
3	API Express (Node.js)	Règles métier
4	Sequelize ORM → MySQL	Persistance

Couche	Technologie	Rôle
Front-end	React (Vite)	Interface, navigation, appels API
Back-end	Node.js + Express	API, règles métier, sessions
Données	MySQL + Sequelize	Stockage relationnel, migrations
Hébergement	o2switch	HTTPS, domaine, app Node

En **développement** : front :5173, API :3000 (CORS + cookie). En **production** : même domaine (`cookileandco.fr`), Express sert le build React et l'API sous `/api/...`

Environnement local : Laragon (MySQL, phpMyAdmin). Deux terminaux tournent en parallèle :

- dossier `server/` → `npm run dev` (API sur le port 3000, connexion MySQL)
- dossier `client/` → `npm run dev` (interface Vite sur le port 5173)

Environnement local — Laragon + npm run dev

Terminal 1 — dossier server/ (API Express)

```
PS server> npm run dev
Connexion à la DB mysql : ok
Server démarré sur http://localhost:3000
```

Terminal 2 — dossier client/ (React + Vite)

```
PS client> npm run dev
VITE ready — http://localhost:5173/
```

Laragon : MySQL + phpMyAdmin en arrière-plan

Environnement local : 2 terminaux (server/ + client/)

3.2 Organisation du front-end

Dossier `client/src/` :

Dossier	Contenu
<code>pages/</code>	HomePage, RecipesPage, RecipeDetailsPage, AdminPage...
<code>components/</code>	Layout, Header, RecipeCard, AdminRoute...
<code>context/</code>	AuthContext, ToastContext
<code>services/</code>	api.js, authServices.js
<code>style/</code>	themes.css (variables Madras)

Routes protégées côté client via `AdminRoute` (en complément du contrôle serveur).

3.3 Organisation du back-end

Couche	Rôle	Exemple
Routes	URLs API + enchaînement middlewares	<code>recipeRoutes.js</code>
Middlewares	Auth, validation, upload, erreurs	<code>requireAdmin</code> , <code>validate</code>
Contrôleurs	Orchestre la réponse HTTP	<code>recipeController.js</code>

Couche	Rôle	Exemple
Validators	Règles <code>express-validator</code>	<code>createRecipeRules.js</code>
Modèles	ORM Sequelize + associations	<code>Recipe.js</code>
Migrations	Schéma versionné	<code>server/migrations/</code>

Dossier `server/` : `app.js`, `server.js`, puis `routes/`, `controllers/`, `middlewares/`, `validators/`, `models/`, `migrations/`, `uploads/`.

3.4 Stack technique et justifications

Couche	Technologie	Justification
Front	React, React Router, Vite	SPA, HMR, build optimisé
Langage	JavaScript (ES modules)	Choix du Projet de Fin d'Année, pas de TypeScript
Back	Node.js, Express	Même langage, API REST claire
Auth	<code>express-session</code> + cookie <code>httpOnly</code>	Session navigateur, adaptée à une SPA sur le même domaine
Validation	<code>express-validator</code>	Messages <code>{ error, fields }</code>
Mots de passe	<code>bcrypt</code>	Jamais en clair en BDD
ORM	Sequelize + MySQL	Relations N-1 / 1-N, migrations
Upload	<code>multer</code>	Photos recettes (type / taille filtrés)
Local	Laragon	MySQL + phpMyAdmin sous Windows
Prod	<code>o2switch</code>	Node + MySQL + HTTPS

3.5 API REST (aperçu)

Méthode	Endpoint	Accès
GET	<code>/api/health</code>	Public
GET	<code>/api/recipes</code>	Public (pagination, filtres)
GET	<code>/api/recipes/:id</code>	Public
POST / PUT / DELETE	<code>/api/recipes...</code>	Admin (session)
POST	<code>/api/auth/login</code>	Public
POST	<code>/api/auth/logout</code>	Session
GET	<code>/api/auth/current-user</code>	Admin
GET / CRUD	<code>/api/categories</code> , <code>/api/origins</code> , <code>/api/meal-types</code>	Lecture publique ; écriture admin

Tests réalisés avec **Postman** (collection *cookîle&co*) : détails § 4.3.

3.6 Sécurité

Mesure	Implémentation
Authentification	<code>express-session</code> , cookie <code>httpOnly</code> , <code>sameSite: 'lax'</code> , <code>secure</code> en HTTPS
Autorisation	Middleware <code>requireAdmin</code> (401 / 403)
Validation	<code>express-validator</code> + <code>validate.js</code>
CORS	Origine limitée à <code>CLIENT_URL</code> + <code>credentials: true</code>
Upload	Filtre type / taille (<code>multer</code>)
Erreurs API	Format <code>{ "error": "...", }</code> , pas de stack en JSON

Configuration session (`server/app.js`, `express-session`) :

Option	Valeur	Rôle
<code>secret</code>	<code>process.env.SESSION_SECRET</code>	Signature du cookie
<code>resave</code>	<code>false</code>	Pas de réécriture inutile de session
<code>saveUninitialized</code>	<code>false</code>	Pas de session vide créée
<code>cookie.httpOnly</code>	<code>true</code>	Cookie illisible en JavaScript (anti-XSS)
<code>cookie.sameSite</code>	<code>'lax'</code>	Limite les envois cross-site
<code>cookie.secure</code>	<code>true</code> en prod HTTPS	Cookie uniquement sur HTTPS

3.7 Hébergement et déploiement (o2switch)

Site en production : <https://www.cookileandco.fr>

Élément	Emplacement
Code (Git)	<code>~/CookIleAndCo/</code>
Config	<code>server/.env</code> sur le serveur
Build React	<code>client/dist/</code>
App Node cPanel	Racine <code>server/</code> , démarrage <code>server.js</code>

Étapes type : `git push` → `git pull` serveur → `npm install` → `.env prod` → migrations → `VITE_API_URL="" npm run build` → redémarrage app Node.

Test post-déploiement	Résultat attendu
Page d'accueil	Site accessible sur https://www.cookileandco.fr
<code>/api/health</code>	JSON statut API (ex. <code>{ "status": "...", }</code>)
Connexion admin	Cookie session <code>httpOnly</code> + <code>secure</code>

Le site est en ligne sur <https://www.cookileandco.fr> ; un test sur `/api/health` confirme que l'API répond bien en production.

4. Réalisations

Ci-dessous, trois extraits du code : protection admin, validation des entrées, et chargement du catalogue côté React.

4.1 Réalisations back-end

4.1.1 Contrôle d'accès : middleware `requireAdmin`

Fichier : `server/middlewares/accessControl.js`

Avant de créer ou modifier une recette, le serveur vérifie qu'une session admin est active. Sans ça, n'importe qui pourrait appeler `POST /api/recipes`.

```
export function requireAdmin(req, res, next) {
  const userId = req.session.userId;
  const role = req.session.role;

  if (!userId) {
    res.status(401).json({ error: 'Non authentifié' });
  } else if (role !== 'admin') {
    res.status(403).json({ error: 'Accès interdit' });
  } else {
    next();
  }
}
```

En pratique : **401** si personne n'est connecté, **403** si le rôle n'est pas admin. Le secret de session est dans le fichier `.env`. Toutes les routes d'écriture réutilisent ce middleware.

La maquette Figma présente l'interface de connexion administrative. Le header est vert foncé avec le logo 'Cook'ile.co' à gauche et les liens 'Accueil', 'Recettes' et 'Catégories' à droite. Le corps principal est beige et contient un formulaire centré intitulé 'Formulaire de connexion'. Ce formulaire comporte deux champs de saisie : 'Email*' avec le placeholder 'Entre ton email' et 'Mot de passe*' avec le placeholder 'Entre ton mot de passe'. En dessous se trouve un bouton orange 'Se connecter'. Le footer est vert foncé et indique les droits d'auteur : '© 2026 Tetelle Cook'ile & Co' et 'By AnthonyDevBdx'.

Maquette Figma : connexion administratrice

4.1.2 Validation des entrées et chaîne de middlewares

Fichiers : `server/validators/createRecipeRules.js`, `server/middlewares/validate.js`,
`server/routes/recipeRoutes.js`

Une recette avec un titre vide ou une clé étrangère manquante ne doit pas atteindre la base. Les règles `express-validator` passent **avant** le contrôleur.

```
// Ordre : droits → règles → validation → contrôleur
router.post('/', requireAdmin, createRecipeRules, validate, createRecipe);
router.put('/:id', requireAdmin, createRecipeRules, validate, updateRecipe);
```

```
// validate.js : si erreurs, 400 + détail des champs
const result = validationResult(req);
if (!result.isEmpty()) {
  res.status(400).json({
    error: 'Validation échouée',
    fields: result.mapped(),
  });
} else {
  next();
}
```

Le front reçoit des messages clairs par champ ; la même logique s'applique à la création et à la modification.

Modifier la recette

Informations générales

Titre *

Temps de cuisson (min) *

Photo

NO IMAGE

Choisir un fichier | Aucun fichier choisi

Ingrédients

Ingrédient 1 *

Quantité *

Unité

Nom *

Supprimer cet ingrédient

Ingrédient 2 *

Quantité *

Unité

Nom *

Supprimer cet ingrédient

+ Ajouter un ingrédient

Préparation

Étape 1 *

Description *

Supprimer cette étape

Étape 2 *

Description *

Supprimer cette étape

+ Ajouter une étape

Conseils de Tellele

Classification

Catégorie

Choisir

Choisir une catégorie

Choisir une catégorie

+ Ajouter

Modifier

Choisir une catégorie dans « Choisir »

Nouveau nom

Modifier

Supprimer

Choisir une catégorie dans « Choisir »

Supprimer

Origine

Choisir

Choisir une origine

Choisir une origine

+ Ajouter

Modifier

Choisir une origine dans « Choisir »

Nouveau nom

Modifier

Supprimer

Choisir une origine dans « Choisir »

Supprimer

Type de repas

Choisir

Choisir un type de repas

Choisir un type de repas

+ Ajouter

Modifier

Choisir un type de repas dans « Choisir »

Nouveau nom

Modifier

Supprimer

Choisir un type de repas dans « Choisir »

Supprimer

[Retour au dashboard](#)

[Mettre à jour la recette](#)

© 2016 Tellele Cook'N & Co
by Anthony Onedda

Maquette Figma : formulaire d'édition recette

4.1.3 Accès aux données : liste paginée Sequelize

Fichier : `server/controllers/recipeController.js`

Le catalogue public renvoie les recettes avec leurs associations (catégorie, origine, type de repas), paginées pour ne pas tout charger d'un coup.

```
const result = await Recipe.findAndCountAll({
  where: recipeWhere,
  order: [['createdAt', 'DESC']],
  limit: limitFromUrl,
  offset,
  include: [categoryInclude, originInclude, mealTypeInclude],
});

res.status(200).json({
  data: result.rows,
  meta: { page, limit, total, totalPages },
});
```

Réponse JSON stable (`data` + `meta`) ; filtres et pagination gérés côté SQL via l'ORM.

4.2 Réalisations front-end

4.2.1 Service API centralisé

Fichier : `client/src/services/api.js`

Tous les appels HTTP passent par le même service, avec le cookie de session et la bonne URL de base.

```
async function requestFromApi(method, path, body) {
  const options = {
    method: method,
    credentials: 'include', // cookie de session admin
    headers: {},
  };
  return fetch(apiBaseUrl + path, options);
}
```

Un seul endroit pour `credentials` et l'URL ; même comportement en local et en prod.

4.2.2 Authentification front : AuthContext et garde admin

Fichiers : `client/src/context/AuthContext.jsx`, `client/src/components/AdminRoute.jsx`

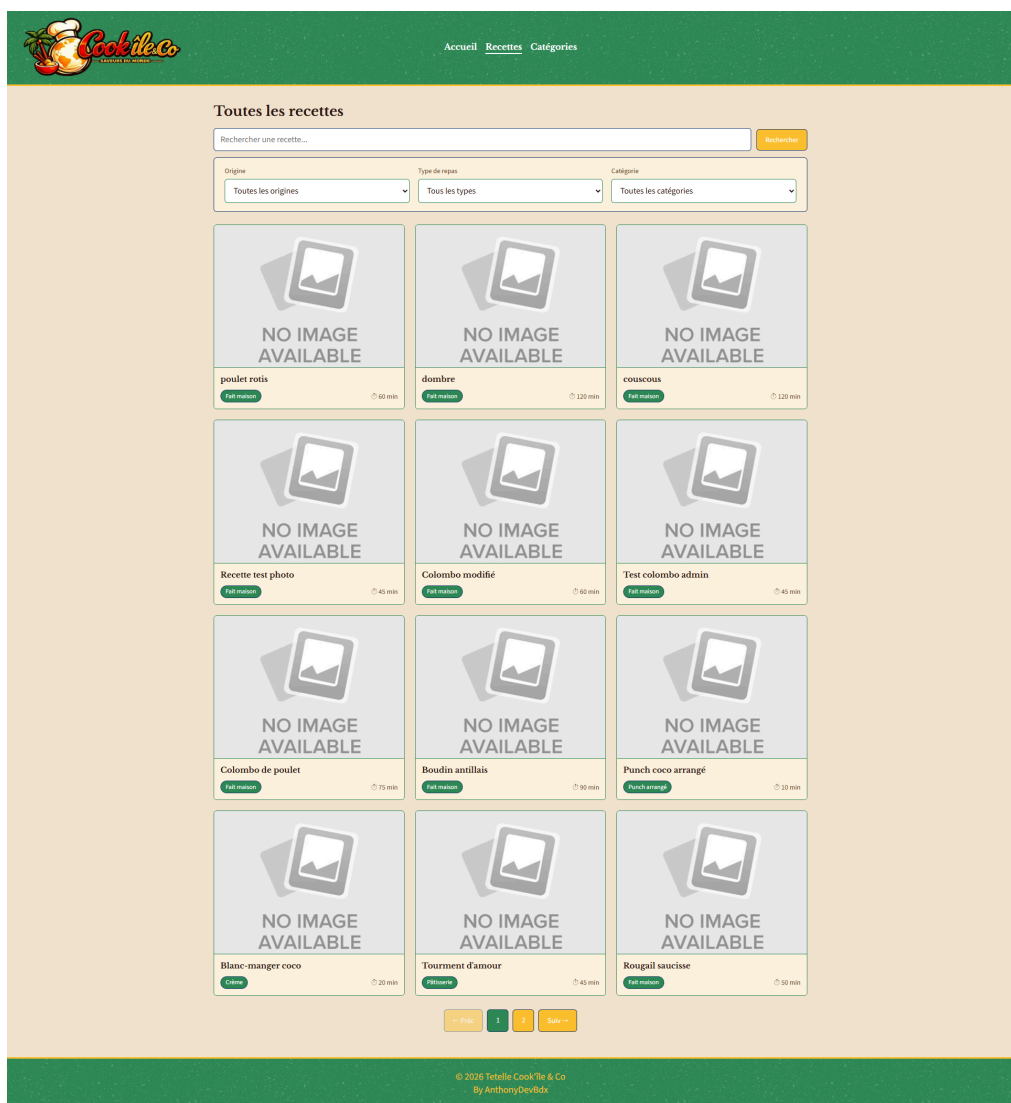
Après login, l'interface doit savoir qui est connecté et bloquer `/admin` sans session : le serveur reste juge en dernier ressort.

Le contexte appelle `/api/auth/current-user` au chargement ; `AdminRoute` redirige vers la connexion si besoin.

4.2.3 Catalogue dynamique et filtres

Fichiers : `client/src/pages/RecipesPage.jsx`, `client/src/components/RecipeCard.jsx`

Le visiteur filtre par origine, type de repas ou catégorie, les résultats se mettent à jour via l'API.



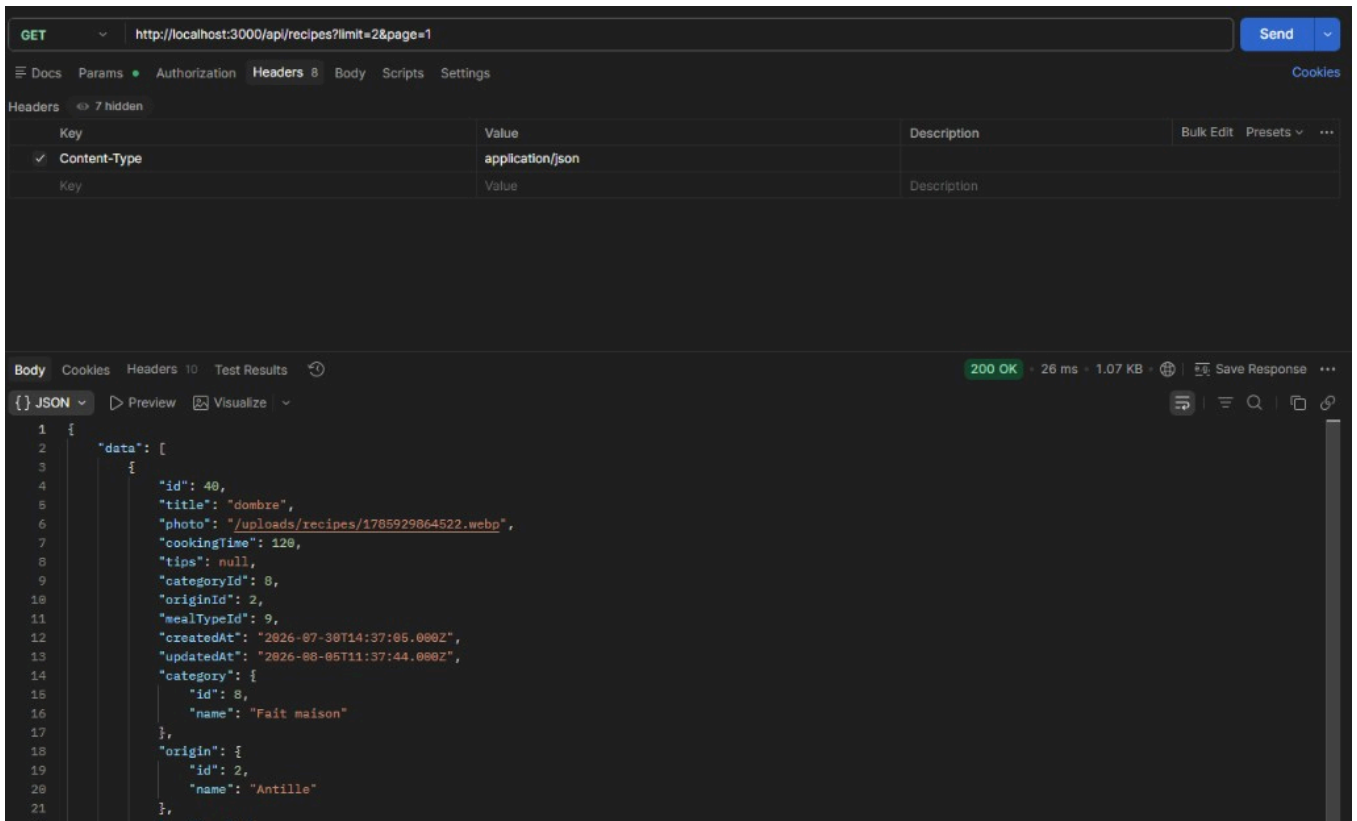
Maquette Figma : catalogue et filtres

Parcours public sans compte ; cartes réutilisables ; rendu aligné sur le thème Madras.

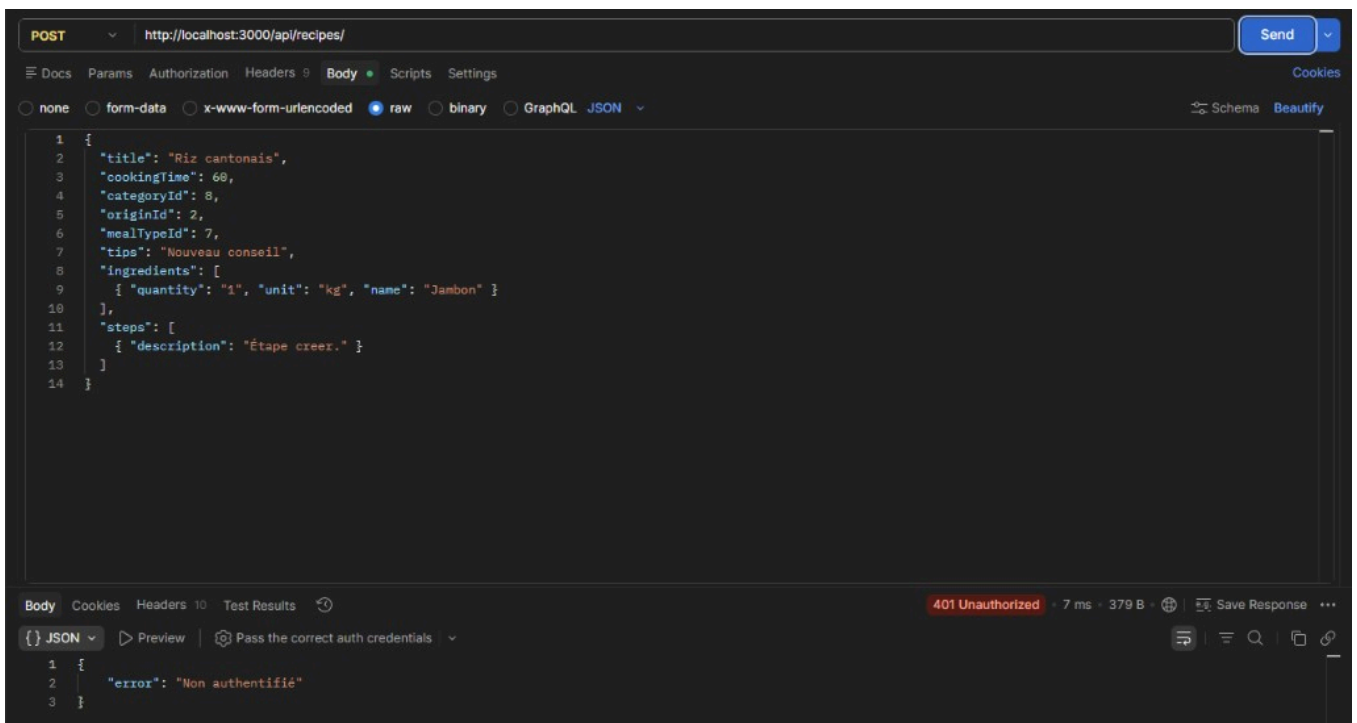
4.3 Preuves de test API (Postman)

Collection Postman **cookile&co**. Quelques requêtes types :

Requête	Méthode / URL	Attendu	Observé
list recettes	GET /api/recipes	200 + JSON paginé	☑
creer recette (sans session)	POST /api/recipes	401 Non authentifié	☑
modifier recette	PUT /api/recipes/:id	200 si cookie admin	☑
health	GET /api/health	200 statut API	☑



Postman : GET /api/recipes



Postman : POST /api/recipes sans session (401)

Sans cookie, la création est refusée : c'est bien `requireAdmin` qui bloque.

4.4 Compétences illustrées

Compétence	Illustration dans cette section
1.3 / 1.4	Composants React, catalogue dynamique, <code>api.js</code>
2.6	Sequelize <code>findAndCountAll</code> + associations
2.7	<code>requireAdmin</code> , validation, routes CRUD
2.8	(déploiement documenté en § 3.7)

5. Jeu d'essai et stratégie de test

Avant de valider une fonctionnalité, je la teste à la main : parcours navigateur pour le métier, Postman pour l'API, phpMyAdmin pour vérifier que la base a bien suivi.

Type	Outil	Ce qu'on vérifie
Jeu d'essai	Navigateur + checklist	Parcours métier, affichage, droits
Test API	Postman	Statuts HTTP, JSON, auth (401/200)
Contrôle BDD	phpMyAdmin	Persistance après modification

Environnement : Chrome, Laragon (MySQL + API :3000 + Vite :5173).

5.1 Contexte du scénario

Fonctionnalité testée : **modification d'une recette par un administrateur**.

Estelle corrige le titre et le temps de cuisson de la recette **id 19** (« Colombo de poulet »). On vérifie l'écran public et la cohérence en base.

5.2 Tableau de test

Étape	Action	Résultat attendu (écran)	Résultat attendu (BDD)	Statut
1	Ouvrir <code>/connexion-superadmin</code> , saisir identifiants admin	Redirection vers <code>/admin</code>	n/a	☑
2	Cliquer modifier sur la recette id 19	Formulaire pré-rempli	n/a	☑
3	Titre → « Colombo de poulet test », temps → 80 min	Champs mis à jour	n/a	☑
4	Enregistrer	Message succès / retour dashboard	Ligne <code>recipes</code> id 19 mise à jour	☑
5	Ouvrir <code>/recettes/19</code> (visiteur)	Nouveau titre et temps affichés	Cohérent avec BDD	☑
6	Vérifier phpMyAdmin, table <code>recipes</code>	n/a	<code>title</code> et <code>cooking_time</code> modifiés	☑

5.3 Illustrations du scénario

Les trois étapes clés : formulaire admin modifié, fiche publique mise à jour, contrôle en base.



Accueil
Recettes
Catégories

Déconnexion

Modifier la recette

Informations générales

Titre *

Contenu de page - recette Teste

Temps de cuisson (min) *

30

Photo

NO IMAGE

Choisir une image

Ingrédients

Ingrédient 1 *

Quantité *

1

Unité

kg

Nom *

potier

Supprimer cet ingrédient

Ingrédient 2 *

Quantité *

2

Unité

g

Nom *

colombo

Supprimer cet ingrédient

Ajouter un ingrédient

Préparation

Etape 1 *

Description *

Paire verser le produit avec origan et ail.

Supprimer cette étape

Etape 2 *

Description *

Apaiser le colombo, les légumes et laisser mijoter.

Supprimer cette étape

Ajouter une étape

Consente de Teste

Laisser mijoter à feu doux pour un colombo fondant.

Classification

Catégorie

Choisir une catégorie

Fait maison

Nouvelle catégorie

Choisir une image

Supprimer

Modifier

Nommez bien

Modifier

Image

NO IMAGE

Choisir une image

Supprimer cette image

Supprimer

Fait maison

Supprimer

Origine

Choisir une origine

Asie

Nouvelle origine

Supprimer

Modifier

Nommez bien

Modifier

Supprimer

Asie

Supprimer

Type de repas

Choisir un type de repas

Déjeuner

Nouvelle type de repas

Supprimer

Modifier

Nommez bien

Modifier

Supprimer

Déjeuner

Supprimer

Ajouter à mes favoris

Retour au dashboard

© 2026 Teste Cool'itea & Co
By AnthonyDevWeb

Jeu d'essai : formulaire modifié



[Accueil](#) [Recettes](#) [Catégories](#)

Déconnexion



NO IMAGE AVAILABLE

Colombo de poulet — recette Tetelle

Fait maison

🕒 80 minutes

Partager cette recette

Facebook · WhatsApp · Copier le lien

Ingrédients

- 1 kg poulet
- 2 c. à s. colombo

Préparation

- 1 Faire revenir le poulet avec oignons et ail.
- 2 Ajouter le colombo, les légumes et laisser mijoter.

Conseils de Tetelle

Laisser mijoter à feu doux pour un colombo fondant.

Commentaires

Votre pseudo

Votre commentaire

Vérification anti-spam

4 + 3 = 7

Résultat

Résous l'addition pour confirmer que tu n'es pas un robot.

Publier

Aucun commentaire pour l'instant. Soyez le premier à réagir !

© 2026 Tetelle Cook'ile & Co
By AnthonyDevBdx

Jeu d'essai : fiche publique

Showing rows 0 - 12 (13 total, Query took 0.0008 seconds)

ELECT * FROM `recipes`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Extra options

	id	title	photo	cooking_time	tips	category_id	origin_id	meal_type_id	created_at	updated_at
	18	Accras de morue	NULL	25	Ne pas tasser la pâte : des accras plus aérés.	5	2	8	2026-07-20 11:26:17	2026-07-20 11:26:17
	19	Colombo de poulet test	NULL	80	Laisser mijoter à feu doux pour un colombo fondant...	8	2	7	2026-07-20 11:26:17	2026-08-06 09:11:47
	20	Boudin antillais	NULL	90	Servir tiède avec du pain et de la céleri-rave.	8	2	9	2026-07-20 11:26:17	2026-07-20 11:26:17
	21	Ti-punch	NULL	5	Utiliser un rhum agricole blanc bien frais.	2	2	9	2026-07-20 11:26:17	2026-07-20 11:26:17
	22	Punch coco arrangé	NULL	10	Laisser reposer 24 h au frais avant de servir.	12	2	9	2026-07-20 11:26:17	2026-07-20 11:26:17
	23	Blanc-manger coco	NULL	20	Filmer au contact pour éviter la peau en surface.	7	2	8	2026-07-20 11:26:17	2026-07-20 11:26:17
	24	Gâteau patate	NULL	60	Attendre le refroidissement complet avant de démou...	4	2	8	2026-07-20 11:26:17	2026-07-20 11:26:17
	26	Pain au lait antillais	NULL	40	Badigeonner de lait chaud à la sortie du four.	11	2	6	2026-07-20 11:26:17	2026-07-20 11:26:17
	27	Sorbet coco maison	NULL	240	Remuer toutes les 30 minutes si pas de sorbetière.	9	2	8	2026-07-20 11:26:17	2026-07-20 11:26:17
	29	Samoussas légumes	NULL	35	Bien égoutter la garniture pour des samoussas crou...	10	3	8	2026-07-20 11:26:17	2026-07-20 11:26:17
	30	Poulet au curry	NULL	55	Faire revenir les épices 1 minute avant le lait de...	8	3	7	2026-07-20 11:26:17	2026-07-20 11:26:17
	31	Nems au porc	NULL	40	Coller les bords avec un mélange farine + eau.	10	3	8	2026-07-20 11:26:17	2026-07-20 11:26:17
	40	dombre	/uploads/recipes/1785929864522.webp	120	NULL	8	2	9	2026-07-30 14:37:05	2026-08-05 11:37:47

☐ Check all

With selected:

Edit

Copy

Delete

Export

☐ Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Jeu d'essai : contrôle en base

5.4 Données observées

Champ	Avant	Après test (BDD)
title	Colombo de poulet	Colombo de poulet test
cooking_time	75	80

5.5 Conclusion du jeu d'essai

Connexion → édition → enregistrement → consultation publique → contrôle BDD : le fil fonctionne. Les tests Postman (§ 4.3) confirment le **401** sans session et la lecture publique en **200**.

6. Veille technologique et sécurité

6.1 Comment j'ai mené la veille

Dès qu'un choix touchait l'auth, les cookies, la validation ou l'upload, je prenais le temps de lire une source fiable (OWASP, doc Express, MDN), de comparer les options, d'appliquer dans le code, puis de retester : Postman pour l'API, F12 pour le cookie, et une passe sur la prod en HTTPS.

6.2 Sources utilisées

Source	Usage
OWASP Session Management Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html	Flags cookie, risques XSS / vol de session
Documentation Express <code>express-session</code> : https://github.com/expressjs/session#cookie	Configuration concrète Node (cookie, session)
MDN : HTTP cookies	<code>httpOnly</code> , <code>Secure</code> , <code>SameSite</code>
Documentation Sequelize / <code>express-validator</code>	Accès données et validation entrées

6.3 Sécurisation des sessions admin

L'espace admin permet de créer, modifier et supprimer des recettes. Après login, le serveur mémorise l'administrateur connecté via **express-session** et un **cookie de session**. Il fallait le configurer correctement : le front React ne doit pas stocker de jeton, et le cookie ne doit pas être lisible ni envoyé n'importe comment.

Risques repérés :

- Cookie **lisible en JavaScript** (sans `httpOnly`) : en cas de faille XSS, un script pourrait voler la session admin.
- Cookie envoyé trop largement (sans `sameSite` / sans `secure` en HTTPS) : envoi cross-site involontaire ou interception hors HTTPS.

Lecture : fiche **OWASP Session Management Cheat Sheet** (sections `httpOnly`, `sameSite`, `secure`), complétée par la doc Express session.

Série de fiches récapitulatives
OWASP

Aide-mémoire pour la gestion des sessions

Introduction

Authentification Web, gestion des sessions et contrôle d'accès :

Une session web est une séquence de transactions HTTP (requêtes et réponses) associées à un même utilisateur. Les applications web modernes et complexes nécessitent la conservation d'informations ou d'un statut concernant chaque utilisateur pendant plusieurs requêtes. Les sessions permettent ainsi de définir des variables, telles que les droits d'accès et les paramètres de localisation, qui s'appliquent à chaque interaction de l'utilisateur avec l'application web durant toute la durée de la session.

Les applications web peuvent créer des sessions pour suivre les utilisateurs anonymes dès leur première requête. Par exemple, elles peuvent conserver la langue préférée de l'utilisateur. De plus, les applications web utilisent les sessions une fois l'utilisateur authentifié. Cela permet de l'identifier lors de ses requêtes ultérieures, d'appliquer des contrôles d'accès sécurisés, d'autoriser l'accès à ses données personnelles et d'améliorer l'expérience utilisateur. Ainsi, les applications web actuelles offrent des fonctionnalités de session avant et après l'authentification.

Une fois la session authentifiée établie, l'identifiant de session (ou jeton) est temporairement équivalent à la méthode d'authentification la plus robuste utilisée par l'application, comme un nom d'utilisateur et un mot de passe, une phrase secrète, un mot de passe à usage unique (OTP), un certificat numérique côté client, une carte à puce ou des données biométriques (empreinte digitale ou rétine). Consultez le [guide d'authentification OWASP](#).

Le protocole HTTP est sans état ([RFC 2616](#), section 5), où chaque paire requête-réponse est indépendante des autres interactions web. Par conséquent, pour introduire la notion de session, il est nécessaire d'implémenter des mécanismes de gestion de session qui lient les modules d'authentification et de contrôle d'accès (ou d'autorisation) généralement disponibles dans les applications web.

Sécurité REST
Ruby on Rails
Sécurité SAML
Prévention des injections SQL
Gestion des secrets
Opérations de modélisation d'IA sécurisées
Architecture cloud sécurisée
Revue de code sécurisé
Programmation sécurisée avec l'IA
Conception de produits sécurisés
Sécurisation des feuilles de style en cascade
Terminologie de sécurité
Prévention de la falsification de requêtes côté serveur
Sécurité FaaS sans serveur
Gestion de session
Sécurité de la chaîne d'approvisionnement logicielle
Prévention des prises de contrôle de sous-domaines
Symphonie
Chaîne de chiffrement TLS
Gestion de JavaScript tiers
Intégration de passerelle de paiement tierce
Modélisation des menaces
Autorisation de transaction
Protection de la couche transport
Sécurité de la couche transport
Redirections et transferts non validés

OWASP : Aide-mémoire gestion des sessions

Mise en place : session serveur uniquement, pas de token côté front.

1. **express-session** avec cookie **httpOnly** (le JS du front ne peut pas lire le cookie).
2. **sameSite: 'lax'** pour limiter les envois cross-site non souhaités.
3. **secure: true** en production HTTPS (`COOKIE_SECURE=true` sur o2switch).
4. **CORS** restrictif : seule `CLIENT_URL` avec `credentials: true`.
5. Middleware **requireAdmin** sur toutes les routes de modification.

Configuration appliquée (`server/app.js`) :

Élément	Réglage
CORS	<code>origin: CLIENT_URL, credentials: true</code>
Session <code>secret</code>	Variable d'environnement <code>SESSION_SECRET</code>
Cookie <code>httpOnly</code>	<code>true</code>
Cookie <code>sameSite</code>	<code>'lax'</code>
Cookie <code>secure</code>	<code>true</code> en production HTTPS

Contrôles :

- `POST /api/recipes` **sans** cookie → **401** `{ "error": "Non authentifié" }`.
 - F12 → Application → cookie de session marqué **HttpOnly**.
-

6.4 Validation et upload

La veille a aussi porté sur :

- **Validation serveur** (`express-validator`) : ne jamais faire confiance au seul front ; format d'erreur `{ error, fields }`.
- **Upload** (`multer`) : filtrer type MIME et taille pour limiter les fichiers malveillants sur les photos de recettes.

Même connectée, l'admin ne peut pas injecter des données invalides ou des fichiers hors politique.

6.5 Bilan

Pour une SPA qui appelle la même API en production, une session serveur avec cookie `httpOnly`, `sameSite` et `secure` en HTTPS me paraît adaptée. Les réglages viennent surtout de la fiche OWASP ; les tests Postman du § 4.3 confirment qu'une création de recette sans session renvoie bien 401.

7. Conclusion et bilan

7.1 En bref

Cook'île & Co est le catalogue de recettes d'Estelle : consultation libre pour les visiteurs, gestion réservée aux administrateurs, identité Madras, en ligne sur <https://www.cookileandco.fr>.

7.2 Ce qui est en place

À l'issue du Projet de Fin d'Année, le site propose :

- un **catalogue public** avec recherche et filtres (origine, type de repas, catégorie) ;
 - des **fiches recettes** complètes (photo, ingrédients, étapes, conseils) ;
 - les **référentiels** catégories, origines et types de repas ;
 - un **espace admin** sécurisé (CRUD recettes, upload photos, gestion des référentiels) ;
 - un **thème Madras** responsive ;
 - un **déploiement** fonctionnel sur o2switch en HTTPS.
-

7.3 Difficultés rencontrées

- **Formulaire recette admin** : saisir et enregistrer ingrédients, étapes et photo dans un seul parcours, avec validation côté serveur.
 - **Associations Sequelize** : relier correctement recette, catégorie, origine, type de repas et tables filles (ingrédients, étapes).
 - **Mise en ligne sur o2switch** : build du front, configuration `.env`, migrations et vérifications après déploiement.
-

7.4 Ce que j'ai appris

Sur ce projet j'ai surtout progressé sur **React** (composants, hooks, routing, contexte d'auth), l'**API Express** (routes, contrôleurs, middlewares, validation), **Sequelize** (modèles, associations, migrations, pagination), et les bases de **sécurité web** (session, bcrypt, contrôle d'accès, CORS). J'ai aussi enchaîné **Laragon** en local et **o2switch** en production.

7.5 Limites du projet

- Pas de tests automatisés : contrôles manuels navigateur + Postman.
 - Pas d'inscription / espace personnel pour les visiteurs (catalogue en lecture libre).
-

7.6 Perspectives

- Inscription / profil utilisateurs.
 - Planning hebdomadaire.
 - Amélioration accessibilité et tests automatisés.
 - Optimisation des images et du temps de chargement mobile.
-

7.7 Conclusion personnelle

Ce Projet de Fin d'Année m'a fait enchaîner les échanges avec Estelle, les maquettes Figma, le MCD, le code front et back, puis la mise en ligne. Cook'île & Co sert vraiment Estelle pour partager ses recettes ; ce n'était pas qu'un exercice scolaire.

Annexes

Lexique technique

Terme	Définition
API REST	Interface HTTP (GET, POST, PUT, DELETE) entre front et serveur.
SPA	Single Page Application : React change le contenu sans recharger toute la page.
React	Bibliothèque d'interface par composants.
Express	Framework Node.js pour l'API et les routes.
Middleware	Fonction exécutée avant le contrôleur (auth, validation...).
ORM / Sequelize	Traduction objets JavaScript ↔ tables MySQL.
Migration	Fichier versionné qui modifie le schéma BDD.
Session	Mécanisme serveur mémorisant qu'un admin est connecté.
Cookie httpOnly	Cookie inaccessible au JavaScript, limite le vol via XSS.
CORS	Règles d'appels API depuis une autre origine.
CRUD	Create, Read, Update, Delete.
bcrypt	Hash des mots de passe en base.
Fetch	API JavaScript pour les requêtes HTTP.
Responsive	Mise en page adaptée mobile / desktop.
Postman	Outil de test des requêtes HTTP de l'API.
MCD	Modèle Conceptuel de Données (entités et associations).